

LLM模型蒸馏与微调实操



>> 今天的学习目标

LLM模型蒸馏与微调实操

- 知识蒸馏核心原理
- 如何 50美金 复刻R1模型

s1: Simple test-time scaling

- Unsloth: LLM高效微调
- CASE: qwen2.5-7B微调 (alpaca-cleaned)
- CASE: 训练自己的R1模型
- CASE: Qwen-VL微调

知识蒸馏核心原理

知识蒸馏核心原理

Thinking: 蒸馏的本质是什么?

知识迁移, 不仅仅是模型压缩

传统理解: 蒸馏 = 把大模型压缩成小模型

更准确的理解: 蒸馏 = 让小模型学到大模型的知识

Thinking: 什么是硬标签, 什么是软标签?

硬标签 (Hard Labels) :

- one-hot编码, 只告诉答案是什么
- 例: 图片分类 [猫=1, 狗=0, 鸟=0]
- 信息量少: 只知道是猫, 不知道其他类别的相似度

软标签 (Soft Labels) :

- 概率分布, 告诉每个选项有多大可能性
- 例: 教师模型输出 [猫=0.8, 狗=0.15, 鸟=0.05]

Thinking: 硬标签和软标签, 哪个信息量更大?

软标签, 猫和狗比猫和鸟更相似

=> 这就是暗知识

温度系数T的作用

Thinking: 温度系数T的作用是什么?

温度系数T是控制概率分布尖锐度的调节器

=> 让教师模型暴露更多暗知识。

假设教师模型看到某张模糊图片，原始输出 logits 为 [猫:8, 狗:2, 鸟:1]:

T=1 (正常) : Softmax 输出 [0.99, 0.01, 0.00]。概率极度向猫倾斜，狗和鸟的信息被抹除 => 学生死记硬背这是猫。

T=5 (高温) : Softmax 输出 [0.64, 0.24, 0.12]。概率被熨平 => 教师透露了 这看起来有点像狗 的细微线索。

=> 高温保留了类别间的相似性关系（猫和狗比猫和鸟更接近），这正是人类专家的直觉。学生模型通过高温蒸馏，不仅学会正确答案，更掌握了知识间的内在联系。

训练阶段使用高T（如3-10）充分传递暗知识；推理阶段回归T=1，恢复模型原本的自信判断，确保输出准确性。

损失函数与LLM蒸馏

蒸馏损失函数：

$$L_{total} = \alpha L_{hard} + (1 - \alpha) L_{soft}$$

蒸馏损失是双师教学：学生既要向标准答案学习，又要模仿教师思路。

L_{hard} : 学生 vs 真实标签的交叉熵，确保基础正确率（如必须认出这是 猫）

L_{soft} : 学生 vs 教师输出的KL散度，学习概率分布间的微妙关系

KL散度：

KL散度衡量两个分布的距离。假设教师对某题输出 [猫:0.7, 狗:0.2, 鸟:0.1]：

若学生输出 [0.6, 0.3, 0.1]，KL值较小，说明学生领悟了"像猫但有点狗的特征"

若学生输出 [0.9, 0.05, 0.05]，KL值较大，虽然答对但丢失了教师的细微判断

Thinking: α 一般取多少适合？

通常取0.5-0.7，保证学生不盲从教师（教师也可能犯错），同时吸收其解题直觉。

如果 $\alpha=1$ 则退化为普通训练，失去蒸馏意义。

LLM时代的蒸馏挑战

Thinking: LLM时代的蒸馏挑战是什么？

1) 教师-学生能力差距过大时的学习障碍

- 72B教师 -> 1.5B学生：差距太大，学生可能学不好
- 类似于：让小学生直接学研究生课程

2) 解决方案：渐进式蒸馏

72B -> 32B -> 7B -> 1.5B

- 每一步差距可控，学生能有效学习
- DeepSeek-R1系列就采用了这种策略

3) 经典蒸馏 vs LLM蒸馏 (API蒸馏)

- 经典蒸馏：需要访问教师模型的内部概率分布
- API蒸馏：只能拿到教师的输出文本 -> 用教师输出文本做SFT
- GPT-5/Claude等闭源模型只能做API蒸馏
- 开源模型（如DeepSeek）可以做经典蒸馏

S1模型就是一个典型的API蒸馏案例

用Gemini（教师模型）生成推理过程 -> 作为SFT训练数据
-> 微调Qwen（学生模型）

李飞飞 50美金 复刻R1模型
s1: Simple test-time scaling

s1: Simple test-time scaling

s1: Simple test-time scaling

<https://arxiv.org/pdf/2501.19393>

- 李飞飞等研究人员用不到50美元的云计算费用，成功训练出了一个名为s1的推理模型。
- 该模型在数学和编码能力中的表现，与OpenAI的O1和DeepSeek-R1等尖端推理模型不相上下。
- s1模型的训练只用了1000个样本数据，具体过程是：使用Gemini对这1000个样本完善推理过程，然后对Qwen模型进行监督微调。

Model	# ex.	AIME 2024	MATH 500	GPQA Diamond
API only				
o1-preview	N.A.	44.6	85.5	73.3
o1-mini	N.A.	70.0	90.0	60.0
o1	N.A.	74.4	94.8	77.3
Gemini 2.0 Flash Think.	N.A.	60.0	N.A.	N.A.
Open Weights				
Qwen2.5- 32B-Instruct	N.A.	26.7	84.0	49.0
QwQ-32B	N.A.	50.0	90.6	65.2
r1	≫800K	79.8	97.3	71.5
r1-distill	800K	72.6	94.3	62.1
Open Weights and Open Data				
Sky-T1	17K	43.3	82.4	56.8
Bespoke-32B	17K	63.3	93.0	58.1
s1 w/o BF	1K	50.0	92.6	56.6
s1-32B	1K	56.7	93.0	59.6

s1: Simple test-time scaling

s1模型方法论：

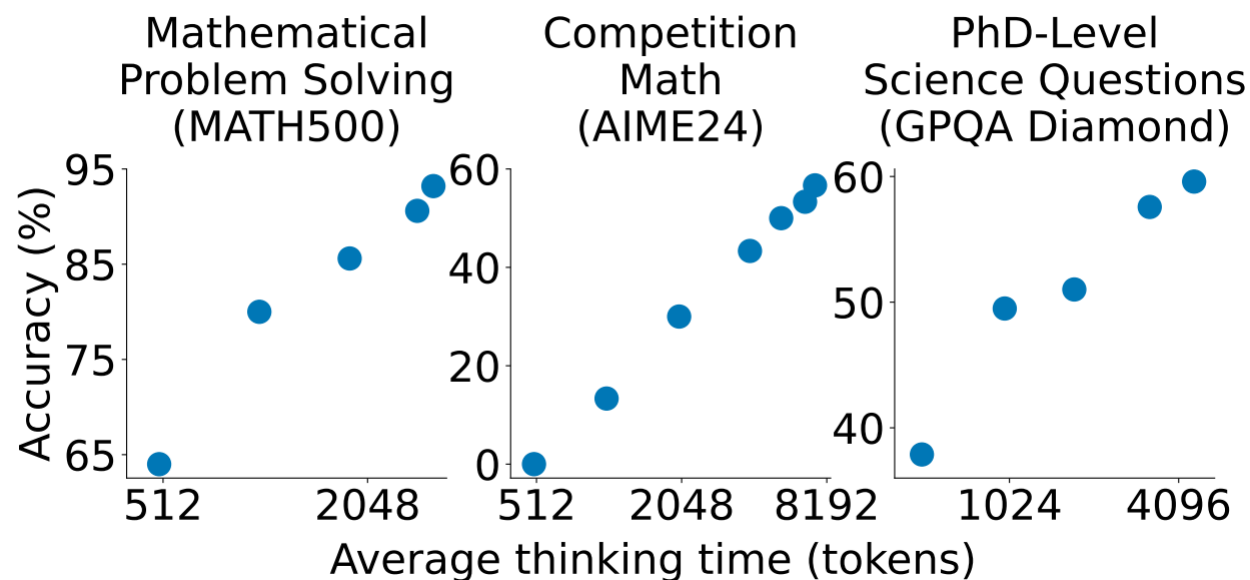
- 数据集构建 (s1K)：

从三个标准——**难度**、**多样性**和**质量**——出发，挑选了1000个问题及其对应的推理路径。

数据集包括来自不同领域，如数学竞赛、物理竞赛，并且新增了两个原创的数据集：s1-prob和s1-teasers，分别涵盖了概率问题和定量交易面试中的难题。

- 预算强制 (Budget Forcing)：

一种控制测试时计算的技术，通过强制终止或延长模型的思考过程（通过添加“Wait”字符串），使模型有机会重新检查答案，从而可能纠正错误的推理步骤。



s1: Simple test-time scaling

预算强制 (Budget Forcing) :

1、设定思考时间限制:

当模型开始处理一个问题时，首先为其设定了一个最大思考时间（以token数量衡量）。如果模型在这个时间内完成了思考并准备给出答案，则按照正常流程进行。

2、强制结束思考过程:

如果模型生成的思考token超过了预设的最大值，系统会强行终止模型的思考过程。这通常是通过添加一个特殊的end-of-thinking token delimiter实现的，促使模型停止进一步的推理，并转向生成最终答案。

3、鼓励更深入的探索:

如果希望模型花更多的时间来考虑一个问题，可以抑制end-of-thinking token delimiter的生成，并在当前的推理路径后面追加“Wait”字符串 => 为了让模型有机会重新评估其先前的推理步骤，可能会纠正一些快速但不准确的回答。

例如，“raspberry”这个单词中有三个'r'，但最初的快速阅读导致模型错误地计数为两个。通过使用“Wait”，模型被引导去重新读取问题和单词，最终正确地计数了三个'r'。

s1: Simple test-time scaling

Thinking: 如何知道每个任务需要多少时间, 如何控制这个 预算限制的方向 (多一些, 少一些) ?

基于训练数据的分析:

通过分析训练集中的问题, 可以估计出不同复杂度的问题所需的平均思考时间。例如, 在实验中, 将推理路径按照长度 (以token数量计) 分组, 并据此对模型进行提示。

基于实验评估的时间调整:

在初步设定的时间预算基础上运行模型, 并记录其在各种问题上的表现。这包括准确性和生成的答案长度等指标。基于此决定增加思考时间, 还是减少思考时间。

使用分类条件控制:

论文提到了一种叫做“class-conditional control”的方法, 它允许用户通过API参数设置不同的“reasoning_effort”级别 (低、中、高) 。虽然这种方法不能精确控制计算量, 但可以根据期望的响应速度和细节水平调整思考时间。

s1: Simple test-time scaling

Thinking: s1整个的训练过程是怎样的？

1、**数据准备**：首先，从大约59,000个问题中精心挑选并构建了一个包含1,000个高质量样本的数据集（s1K）。这些样本包含了问题、由Gemini 2.0 Flash Thinking API生成的推理路径以及对应的答案。

2、**模型选择与微调**：选择了Qwen2.5-32B-Instruct作为基础模型，并在其上进行了监督微调SFT。这里是将s1K数据集中的问题及其对应的推理路径和答案输入到Qwen2.5-32B-Instruct模型中进行训练。

3、**预算强制技术的应用**：在测试阶段，为了控制模型的计算资源使用，采用了“预算强制”技术。

当模型生成的思考token超过设定限制时，通过添加结束思考的标记强制终止思考过程。

如果希望模型花费更多时间思考，则通过追加“Wait”字符串鼓励模型进一步探索。

整个训练是在16个H100 GPU上进行的，耗时约26分钟。

s1: Simple test-time scaling

<|im_start|>system

You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

<|im_end|>

<|im_start|>user

How many r in raspberry

<|im_end|>

<|im_start|>assistant

<|im_start|>think

让我一步步分析：

1. 先写出单词：raspberry
2. 数一下字母r的出现次数：
 - 第一个r在开头
 - 第二个r在中间

- 一共有2个r

<|im_end|>

The word "raspberry" contains 2 letter "r".

<|im_end|>

思考过程控制：

MAX_TOKENS_THINKING = 32000 # 控制思考token的上限

NUM_IGNORE = 1 # 控制思考的次数

通过prompt template引导模型输出思考过程，通过sft.py进行训练，损失函数loss的计算包括：思考过程(<|im_start|>think后的内容)和最终答案

Summary

Thinking: 蒸馏 + SFT + RLHF 在项目中如何使用?

实际项目中，蒸馏、SFT、强化学习往往不是孤立使用，而是分阶段组合：

Phase 1: SFT蒸馏 (学格式+知识)

- 目标：让学生模型学会基本的格式遵循和领域知识
- 数据：教师模型生成的高质量回答 (API蒸馏数据)
- 训练方式：标准SFT，使用教师输出作为标签
- 效果：模型能回答问题，但推理能力有限

Phase 2: GRPO强化学习 (学推理策略)

- 目标：在SFT基础上进一步提升推理能力
- 数据：只需要问题+标准答案 (如GSM8K)
- 训练方式：GRPO，通过奖励函数引导推理
- 效果：模型学会展示推理过程，答案准确率提升

Summary

Thinking: DeepSeek-R1-Distill系列的训练路径是怎样的?

1. 用DeepSeek-R1 (671B) 作为教师模型

2. 生成大量带推理过程的数据

3. 对Qwen2.5系列模型做SFT蒸馏

-> DeepSeek-R1-Distill-Qwen-1.5B/7B/14B/32B

4. 部分模型再做GRPO进一步优化

Summary

Thinking: 教师-学生模型选择原则是什么?

教师模型越强越好 (但API成本要考虑)

学生模型大小根据部署环境选择

- 端侧部署: 1.5B-3B
- 服务器部署: 7B-14B
- 高性能需求: 32B+

师生参数量比一般控制在4:1到10:1之间

CASE: qwen2.5-7B微调
(alpaca-cleaned)

CASE: Qwen2.5-7B微调 (alpaca-cleaned)

yahma/alpaca-cleaned 数据集

- Stanford发布的原始 Alpaca 数据集的清理版本，主要用于LLM的指令微调（Instruct Tuning）。
- 原始数据集：Alpaca-52k，最初用于微调LLaMA模型以生成Alpaca-7B模型，包含约52K条指令数据。
- 清理版本：yahma/alpaca-cleaned 是对原Alpaca数据集的优化版本，修复了数据中的瑕疵（如噪声、格式问题）

Alpaca 数据集是一个用于语言模型指令微调（Instruction Tuning）的数据集，由Stanford研发团队开发

2. 数据格式（JSON格式）

- instruction：用户输入的指令（必填）。
- input：可选的上下文或补充输入（选填）。
- output：模型应生成的回答（必填）

```
{
  "instruction": "计算这些物品的总费用。",
  "input": "汽车 - $3000, 衣服 - $100, 书 - $20。",
  "output": "汽车、衣服和书的总费用为 $3000 + $100 + $20 = $3120。"
}
```

TRL SFTTrainer工具

TRL SFTTrainer工具

- Hugging Face 的 TRL (Transformer Reinforcement Learning) 库中的 SFTTrainer (Supervised Fine-tuning Trainer) 是专门设计用于简化 Transformer 模型监督微调过程的工具。
- SFTTrainer 提供了一系列专门为监督微调设计的功能：

内存高效训练：支持参数高效微调技术(PEFT)，如 LoRA (Low-Rank Adaptation)，可以显著减少内存使用量。

数据集处理简化：自动处理常见的数据格式，简化了从原始文本到模型输入的转换过程。它内置了对文本补全任务的支持，这是许多语言模型微调场景中的常见需求。

训练优化：提供了针对语言模型训练的特定优化，如打包(packing)技术，可以将多个短序列合并成一个长序列，提高训练效率。

灵活配置：保留了原始 Trainer 的所有功能，同时添加了针对监督微调的特定参数和选项

CASE: Qwen2.5-7B微调 (alpaca-cleaned)

```
from unsloth import FastLanguageModel

# 设置模型参数
max_seq_length = 2048 # 设置最大序列长度, 支持 RoPE 缩放
dtype = None # 数据类型, None 表示自动检测。Tesla T4 使用 Float16,
Ampere+ 使用 Bfloat16
load_in_4bit = True # 使用 4bit 量化来减少内存使用

# 加载预训练模型和分词器
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "/root/autodl-tmp/models/Qwen/Qwen2___5-7B-Instruct",
    # 使用Qwen2.5-7B模型
    max_seq_length = max_seq_length,
    dtype = dtype,
    load_in_4bit = load_in_4bit,
)
```

```
# 添加LoRA适配器, 只需要更新1-10%的参数
model = FastLanguageModel.get_peft_model(
    model,
    r = 16, # LoRA秩, 建议使用8、16、32、64、128
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj"], # 需要应用LoRA的模块
    lora_alpha = 16, # LoRA缩放因子
    lora_dropout = 0, # LoRA dropout率, 0为优化设置
    bias = "none", # 偏置项设置, none为优化设置
    use_gradient_checkpointing = "unsloth", # 使用unsloth的梯度检查点, 可减少30%显存使用
    random_state = 3407, # 随机种子
    use_rslora = False, # 是否使用rank stabilized LoRA
    loftq_config = None, # LoftQ配置
)
```

通过FastLanguageModel.from_pretrained() 加载基础模型, 然后通过FastLanguageModel.get_peft_model() 添加 LoRA 层

CASE: Qwen2.5-7B微调 (alpaca-cleaned)

定义Alpaca格式的提示模板

```
alpaca_prompt = """Below is an instruction that describes a task, paired  
with an input that provides further context. Write a response that  
appropriately completes the request.
```

```
### Instruction:
```

```
{}
```

```
### Input:
```

```
{}
```

```
### Response:
```

```
{}"""
```

获取结束标记

```
EOS_TOKEN = tokenizer.eos_token
```

定义数据格式化函数

```
def formatting_prompts_func(examples):
```

```
    instructions = examples["instruction"]
```

```
    inputs       = examples["input"]
```

```
    outputs      = examples["output"]
```

```
    texts = []
```

```
    for instruction, input, output in zip(instructions, inputs, outputs):
```

```
        # 必须添加EOS_TOKEN, 否则生成会无限继续
```

```
        text = alpaca_prompt.format(instruction, input, output) + EOS_TOKEN
```

```
        texts.append(text)
```

```
    return { "text" : texts, }
```

CASE: Qwen2.5-7B微调 (alpaca-cleaned)

加载Alpaca数据集

```
from datasets import load_dataset
```

```
dataset = load_dataset("/root/autodl-tmp/datasets/yahma/alpaca-cleaned", split = "train")
```

```
dataset = dataset.map(formatting_prompts_func, batched = True,)
```

设置训练参数和训练器

```
from trl import SFTTrainer
```

```
from transformers import TrainingArguments
```

```
from unsloth import is_bfloat16_supported
```

定义训练参数

```
training_args = TrainingArguments(
```

```
    per_device_train_batch_size = 2, # 每个设备的训练批次大小
```

```
    gradient_accumulation_steps = 4, # 梯度累积步数
```

```
    warmup_steps = 5, # 预热步数
```

```
    max_steps = 60, # 最大训练步数
```

```
    learning_rate = 2e-4, # 学习率
```

```
    fp16 = not is_bfloat16_supported(), # 是否使用FP16
```

```
    bf16 = is_bfloat16_supported(), # 是否使用BF16
```

```
    logging_steps = 1, # 日志记录步数
```

```
    optim = "adamw_8bit", # 优化器
```

```
    weight_decay = 0.01, # 权重衰减
```

```
    lr_scheduler_type = "linear", # 学习率调度器类型
```

```
    seed = 3407, # 随机种子
```

```
    output_dir = "outputs", # 输出目录
```

```
    report_to = "none", # 报告方式
```

```
)
```

CASE: Qwen2.5-7B微调 (alpaca-cleaned)

创建SFTTrainer实例

```
trainer = SFTTrainer(  
    model = model,  
    tokenizer = tokenizer,  
    train_dataset = dataset,  
    dataset_text_field = "text",  
    max_seq_length = max_seq_length,  
    dataset_num_proc = 2,  
    packing = False, # 对于短序列可以设置为True, 训练速度提升5倍  
    args = training_args,  
)
```

开始训练

```
trainer_stats = trainer.train()
```

Unsloth: Will smartly offload gradients to save VRAM!

 [60/60 01:29, Epoch 0/1]

Step Training Loss

1	1.433300
2	1.786000
3	1.542800
4	1.674900
5	1.452400
6	1.193900
7	0.900700
8	1.154300
9	1.102500
10	1.040300
11	0.844500
12	0.949100
13	0.824900
14	1.035500
15	0.806500

CASE: Qwen2.5-7B微调 (alpaca-cleaned)

模型推理部分

```
FastLanguageModel.for_inference(model) # 启用原生2倍速推理
inputs = tokenizer(
[
    alpaca_prompt.format(
        "Continue the fibonnaci sequence.", # 指令
        "1, 1, 2, 3, 5, 8", # 输入
        "", # 输出留空用于生成
    )
], return_tensors = "pt").to("cuda")

# 生成输出
outputs = model.generate(**inputs, max_new_tokens = 64, use_cache = True)
tokenizer.batch_decode(outputs)
```

****inputs:** 经过 tokenizer 处理后的输入数据
max_new_tokens = 64,
控制模型最多可以生成的新 token 数量
这里设置为 64, 最多会生成 64 个新的 token
如果生成过程中遇到终止符 (如 EOS token), 会提前停止

use_cache = True:
启用 KV cache (键值缓存) 机制
可以显著提升生成速度, 因为不需要重复计算之前的 token 的注意力权重

['Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.\n\n#### Instruction:\nContinue the fibonnaci sequence.\n\n#### Input:\n1, 1, 2, 3, 5, 8\n\n#### Response:\nThe next numbers in the Fibonacci sequence after 1, 1, 2, 3, 5, 8 are:\n\n13, 21, 34, 55, 89, ... \n\nTo generate each number in the sequence, you add the two preceding ones. So starting']

CASE: Qwen2.5-7B微调 (alpaca-cleaned)

微调模型保存

```
model.save_pretrained("lora_model")  
tokenizer.save_pretrained("lora_model")
```

```
('lora_model/tokenizer_config.json',  
'lora_model/special_tokens_map.json',  
'lora_model/vocab.json',  
'lora_model/merges.txt',  
'lora_model/added_tokens.json',  
'lora_model/tokenizer.json')
```



📁 / ... / unsloth_cases / lora_model /

名称	修改时间
📄 adapter_config.json	2 天前
📄 adapter_model.safet...	2 天前
📄 added_tokens.json	2 天前
📄 merges.txt	2 天前
📄 README.md	2 天前
📄 special_tokens_map.j...	2 天前
📄 tokenizer_config.json	2 天前
📄 tokenizer.json	2 天前
📄 vocab.json	2 天前

CASE: 训练自己的R1模型

CASE：训练自己的R1模型

1. 安装和初始化

```
# 安装必要的库
!pip install unsloth vllm

# 导入FastLanguageModel和PatchFastRL
from unsloth import FastLanguageModel, PatchFastRL

PatchFastRL("GRPO", FastLanguageModel)
```

FastLanguageModel，是 unsloth 库提供的一个优化过的 LLM加载器和处理器，主要功能：

- 提供了更快的模型加载和推理速度
- 支持4bit量化加载模型
- 集成了vLLM进行快速推理
- 支持LoRA (Low-Rank Adaptation) 微调

2. 模型加载

```
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "meta-llama/meta-Llama-3.1-8B-Instruct",
    max_seq_length = max_seq_length,
    load_in_4bit = True, # 使用4bit量化加载
    fast_inference = True, # 启用vLLM快速推理
    max_lora_rank = lora_rank,
    gpu_memory_utilization = 0.6, # 控制GPU内存使用
)
```

PatchFastRL，是一个用于强化学习(RL)的补丁函数：

- 为语言模型添加强化学习相关的功能
- 特别支持GRPO (Guided Reward Policy Optimization)算法
- 优化模型训练过程

CASE：训练自己的R1模型

3. 数据准备

使用GSM8K数据集进行训练

定义了特定的系统提示格式：

```
SYSTEM_PROMPT = """  
Respond in the following format:  
<reasoning>  
...  
</reasoning>  
<answer>  
...  
</answer>  
"""
```

4. 奖励函数设计

定义了多个奖励函数来评估模型输出：

- correctness_reward_func: 检查答案正确性
- int_reward_func: 检查输出是否为整数
- strict_format_reward_func: 严格检查输出格式
- soft_format_reward_func: 宽松检查输出格式
- xmlcount_reward_func: 检查XML标签的正确使用

GSM8K数据集

GSM8K (Grade School Math 8K) 是一个高质量的小学数学应用题数据集，主要用于评估和训练人工智能模型在数学推理和多步问题解决方面的能力 <https://huggingface.co/datasets/openai/gsm8k>

question string · lengths	answer string · lengths
 42 985	 50 1.23k
Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell...	Natalia sold $48/2 = \ll 48/2=24 \gg 24$ clips in May. Natalia sold $48+24 = \ll 48+24=72 \gg 72$ clips altogether in April and May. ##### 72
Weng earns \$12 an hour for babysitting. Yesterday, she just did 50 minutes of babysitting. How much did she earn?	Weng earns $12/60 = \$\ll 12/60=0.2 \gg 0.2$ per minute. Working 50 minutes, she earned $0.2 \times 50 = \$\ll 0.2*50=10 \gg 10$. ##### 10
Betty is saving money for a new wallet which costs \$100. Betty has only half of the money she needs. Her parents decided to giv...	In the beginning, Betty has only $100 / 2 = \$\ll 100/2=50 \gg 50$. Betty's grandparents gave her $15 * 2 = \$\ll 15*2=30 \gg 30$. This...
Julie is reading a 120-page book. Yesterday, she was able to read 12 pages and today, she read twice as many pages as yesterday. I...	Maila read $12 \times 2 = \ll 12*2=24 \gg 24$ pages today. So she was able to read a total of $12 + 24 = \ll 12+24=36 \gg 36$ pages since yesterday...
James writes a 3-page letter to 2 different friends twice a week. How many pages does he write a year?	He writes each friend $3*2=\ll 3*2=6 \gg 6$ pages a week So he writes $6*2=\ll 6*2=12 \gg 12$ pages every week That means he writes $12*52=...$
Mark has a garden with flowers. He planted plants of three different colors in it. Ten of them are yellow, and there are 80...	There are $80/100 * 10 = \ll 80/100*10=8 \gg 8$ more purple flowers than yellow flowers. So in Mark's garden, there are $10 + 8 =...$
< Previous 1 2 3 ... 75 Next >	

CASE：训练自己的R1模型

5. 训练配置

```
training_args = GRPOConfig(  
    use_vllm = True,  
    learning_rate = 5e-6,  
    max_steps = 250,  
    save_steps = 250,  
    # 其他训练参数...  
)
```

6. 模型训练

```
trainer = GRPOTrainer(  
    model = model,  
    processing_class = tokenizer,  
    reward_funcs = [  
        xmlcount_reward_func,  
        soft_format_reward_func,  
        strict_format_reward_func,  
        int_reward_func,  
        correctness_reward_func,  
    ],  
    args = training_args,  
    train_dataset = dataset,  
)  
trainer.train()
```

CASE：训练自己的R1模型

7. 模型保存和导出

提供了多种保存格式：

- 16位浮点数格式
- 4位量化格式
- LoRA适配器格式
- GGUF格式（用于llama.cpp）

8. 推理示例

```
text = tokenizer.apply_chat_template([
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": "Calculate pi."},
], tokenize = False, add_generation_prompt = True)
```

用于训练进行结构化推理的大模型，通过特定的格式（reasoning和answer标签）来输出推理过程和最终答案。

- 使用了GRPO训练方法
- 采用了LoRA进行高效微调
- 支持多种量化和保存格式
- 包含了详细的奖励函数设计

GRPO 学习

GRPO (Group Relative Policy Optimization)

- **组相对策略优化**，是一种用于训练LLM的强化学习算法，是DeepSeek-R1模型的核心技术之一
- GRPO的核心在于通过组内样本的相对奖励来优化策略模型，而不是依赖传统的价值函数模型（如PPO中的批评家模型）。它通过采样一组输出，利用这些输出的奖励值来计算相对优势，从而简化了训练过程。

工作原理：

- **采样与奖励计算**：对于每个输入问题，GRPO从当前策略中采样一组输出，并计算每个输出的奖励值。
- **相对优势估计**：通过将每个输出的奖励值与组内平均奖励值进行比较，计算出每个输出的相对优势。
- **策略更新**：根据相对优势，GRPO更新策略模型，优先选择相对优势更高的输出。同时，它通过KL散度约束来控制策略更新的幅度，确保策略分布的稳定性

在DeepSeek-R1中，GRPO被用于优化模型在数学推理、代码生成等复杂任务中的表现。通过生成多个候选解并比较其表现，GRPO能够显著提升模型的推理能力和性能

GRPO 学习

特性	GRPO	PPO
价值网络	无需价值网络，直接使用组内奖励计算相对优势	依赖价值网络估计优势函数，计算复杂度高
奖励计算	通过组内奖励归一化计算相对优势	使用广义优势估计（GAE），依赖未来奖励的折扣总和
策略更新	使用KL散度约束，直接在损失函数中加入KL散度项	使用裁剪概率比，限制策略更新幅度
计算效率	显著提高，降低内存占用	计算效率较低，内存占用大
稳定性	通过组内相对奖励减少策略更新方差	通过裁剪概率比保持训练稳定性

GRPO学习优势：

- 计算效率高：避免了价值网络的训练和维护，显著降低了内存占用和计算开销。
- 稳定性强：通过组内相对奖励减少策略更新的方差，确保训练过程更加稳定。
- 适用性强：特别适合大规模语言模型的微调，尤其在推理任务中表现出色

GRPO 如何引导形成推理能力

Thinking: GRPO可以帮助产生 <reasoning>么? 因为R1模型是推理模型, 所以会有 <reasoning> </reasoning> 这种, 因为数据集 GSM8K只有question, answer两列, 不包含<reasoning>, 那<reasoning>训练的数据是哪里来的?

GRPO是一种引导式奖励策略优化方法, 它通过多个奖励函数引导模型生成特定格式的内容

奖励函数设计:

```
reward_funcs = [  
    xmlcount_reward_func,    # 检查XML标签的正确使用  
    soft_format_reward_func, # 检查是否包含<reasoning>和<answer>标签  
    strict_format_reward_func, # 严格检查格式  
    int_reward_func,         # 检查答案是否为数字  
    correctness_reward_func, # 检查最终答案是否正确  
]
```

GRPO 如何引导形成推理能力

关于<reasoning>的训练，GSM8K确实只有问题和答案，但GRPO不需要显式的reasoning训练数据

它通过奖励函数来引导模型生成reasoning:

```
def soft_format_reward_func(completions, **kwargs) -> list[float]:  
    """检查输出是否包含正确的格式"""  
    pattern = r"<reasoning>.*?</reasoning>\s*<answer>.*?</answer>"  
    responses = [completion[0]["content"] for completion in completions]  
    matches = [re.match(pattern, r) for r in responses]  
    return [0.5 if match else 0.0 for match in matches]
```

训练过程中:

- 模型会生成多个候选输出 (num_generations = 6)
- 每个输出都会被所有奖励函数评估
- 高奖励的输出会被用来指导模型改进生成策略

模型逐渐学会:

1. 使用正确的XML标签格式
2. 在<reasoning>中展示解题步骤
3. 在<answer>中给出正确答案

Llama模型本身已经具备推理能力，GRPO通过奖励机制引导模型，将内部的推理过程显式地写出来，使用特定的格式（XML标签）组织输出，确保最终答案的正确性

Summary

Thinking: GRPO训练与SFT的关键差异是什么?

1. 模型加载需启用vLLM

```
model = FastLanguageModel.from_pretrained(  
    ..., fast_inference=True, gpu_memory_utilization=0.6  
)  
  
# GRPO需要多次生成候选答案, 可以使用vLLM加速推理
```

2. 数据格式: GSM8K只有question和answer

系统提示词要求输出格式, 但训练数据没有reasoning

```
SYSTEM_PROMPT =
```

```
"<reasoning>...</reasoning><answer>...</answer>"
```

3. 奖励函数链 (按顺序评估, 分数累加)

```
reward_funcs = [  
    xmlcount_reward_func,    # XML标签完整性 0.5分  
    soft_format_reward_func, # 有reasoning+answer 0.5分  
    strict_format_reward_func, # 格式严格符合 0.5分  
    int_reward_func,         # 答案为整数 0.5分  
    correctness_reward_func, # 答案正确 2.0分  
]
```

最高可得4.0分, 模型通过梯度更新学习 高奖励=好输出

Summary

4. 训练配置

GRPOConfig(

num_generations=6, # 每个问题生成6个候选, 取相对优势更新

max_steps=250,

learning_rate=5e-6, # 比SFT低一个数量级, 强化学习需稳定

)

5. 保存方式

model.save_lora("grpo_saved_lora") # GRPO专用保存格式

Thinking: 为什么GSM8K没有reasoning却能训练出推理能力?

模型本身有推理能力, 奖励函数只是引导它把推理过程写出来。格式正确+答案正确=高奖励, 模型逐渐学会两者兼顾。

AI大模型趋势

1、小模型将成为主流（大模型蒸馏）

通过蒸馏技术将大型模型的推理能力成功迁移到小型模型中，显著提升了小型模型的性能。

DeepSeek-R1-Distill-Qwen-7B在AIME 2024竞赛中击败了32B模型

2、使用合成数据进行训练成为主流

蒸馏数据是公开的秘密，但很多表现不是蒸馏能解释的。比如v3的中文能力，很多用词和表达方式非常接地气，可能是用了数据合成方法做的预训练。

3、AI模型将自我迭代：强化学习新范式

让模型自己出题自己做，自己检查。

Step1, 模型自己出题自己做，比如出了1亿道题目

Step2, 模型自己检查对不对

Step3, 筛选验证对的内容，将结果与思维链合成新的数据

比如1亿道题目中，自己检查后发现有100万道能验证是对的。那么解出这100万道题的思维链就成了新的训练数据。

不断迭代，探索出之前人类没有探索到的地方（类似AlphaGo-Zero）

Qwen-VL微调

车辆里程表识别

你是一名汽车保险承保专家。这里有一张车辆里程表的图片。请从中提取关键信息。



这张车辆里程表图片显示了以下关键信息：

- 1. **总里程**： 528,915公里
- 2. **当前速度**： 0公里/小时
- 3. **当前时间**： 19:18
- 4. **当前温度**： +45°C
- 5. **当前挡位**： 空挡（N）

这些信息对于评估车辆的使用情况和保险风险非常重要。

车辆里程表识别

你是一名汽车保险承保专家。这里有一张车辆里程表的图片。请从中提取关键信息。



从这张车辆里程表的图片中，可以提取以下关键信息：

1. **当前速度**：56.6英里/小时。
2. **总里程**：37756英里。
3. **当前时间**：11:14。
4. **当前温度**：10.5°C。
5. **当前挡位**：D（前进挡）。

这些信息对于汽车保险承保专家来说，可以帮助评估车辆的使用情况和风险。

CASE： Qwen2.5-VL 3B 视觉模型微调

TO DO：进行汽车保险承保专家的视觉模型微调，实现从车辆里程表图片中提取关键信息的功能。

Step1, 模型加载

```
model, tokenizer = FastVisionModel.from_pretrained(  
    "/root/autodl-tmp/models/Qwen/Qwen2.5-VL-3B-Instruct",  
    use_gradient_checkpointing = "unsloth" # 梯度检查点节省显存  
)
```

Step2, ### 2. LoRA参数配置

```
model = FastVisionModel.get_peft_model(  
    model,  
    finetune_vision_layers = True,    # 微调视觉层  
    finetune_language_layers = True,  # 微调语言层  
    finetune_attention_modules = True, # 微调注意力模块  
    finetune_mlp_modules = True,     # 微调MLP模块  
  
    r = 16,        # LoRA秩  
    lora_alpha = 16, # LoRA alpha参数  
    lora_dropout = 0,  
    bias = "none",  
    random_state = 3407  
)
```

CASE：Qwen2.5-VL 3B 视觉模型微调

训练集Excel准备

列名	说明	示例
id	样本编号	1
prompt	用户提示	你是一名汽车保险承保专家...
image	图片路径	images/1-vehicle-odometer-reading.jpg
response	期望回复	这张车辆里程表图片显示了以下关键信息...

数据转换流程

- 加载Excel文件：使用 `pandas.read\_excel()` 读取数据
- 图片处理：使用 `PIL.Image.open()` 加载RGB格式图片
- 格式转换：转换为标准的消息格式

```
conversation = {
  "messages": [
    {
      "role": "user",
      "content": [
        {"type": "text", "text": prompt},
        {"type": "image", "image": image}
      ]
    },
    {
      "role": "assistant",
      "content": [
        {"type": "text", "text": response}
      ]
    }
  ]
}
```

CASE: Qwen2.5-VL 3B 视觉模型微调

SFT训练器设置

```
trainer = SFTTrainer(  
    model=model,  
    tokenizer=tokenizer,  
    data_collator=UnslothVisionDataCollator(model, tokenizer),  
    train_dataset=converted_dataset,  
    args=SFTConfig(  
        per_device_train_batch_size=2,  
        gradient_accumulation_steps=4,  
        warmup_steps=5,  
        max_steps=30, # 可改为 num_train_epochs=1  
        learning_rate=2e-4,  
        logging_steps=1,  
        optim="adamw_8bit",
```

```
weight_decay=0.01,  
    lr_scheduler_type="linear",  
    seed=3407,  
    output_dir="outputs",  
  
    # 视觉微调必需配置  
    remove_unused_columns=False,  
    dataset_text_field="",  
    dataset_kwargs={"skip_prepare_dataset": True}  
),  
    max_seq_length=2048  
)
```

批次设置: `batch\_size=2` × `accumulation=4` = 有效批次大小8

-学习率: `2e-4` 适合LoRA微调

优化器: `adamw\_8bit` 节省显存

序列长度: `2048` 支持长文本和图片

CASE: Qwen2.5-VL 3B 视觉模型微调

Unsloth: Failed to patch Gemma3ForConditionalGeneration.

🦥 Unsloth Zoo will now patch everything to make training faster!

```
INFO 09-03 18:51:52 __init__.py:183] Automatically detected platform cuda.
```

正在加载Qwen2.5-VL-3B模型...

```
==((====))== Unsloth 2025.3.19: Fast Qwen2 5 Vll patching. Transformers: 4.51.3. vLLM: 0.7.0.
```

```
\\  /| NVIDIA GeForce RTX 4090. Num GPUs = 1. Max memory: 23.65 GB. Platform: Linux.
```

```
0^0/ \_/ \    Torch: 2.5.1+cu121. CUDA: 8.9. CUDA Toolkit: 12.1. Triton: 3.1.0
```

```
\ / Bfloat16 = TRUE. FA [Xformers = 0.0.28.post3. FA2 = True]
```

"- -" Free license: <http://github.com/unslothai/unsloth>

Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!

Loading checkpoint shards: 100% 2/2 [00:07<00:00, 3.88s/it]

Using a slow image processor as ``use_fast`` is unset and a slow processor was saved with this model. ``use_fast=True`` will be the default behavior in v4.52, even if the model was saved with a slow processor. This will result in minor differences in outputs. You'll still be able to use a slow processor with ``use_fast=False``.

配置LoRA参数...

加载训练数据...

Excel文件列名: ['id', 'prompt', 'image', 'response']

数据集形状: (2, 4)

成功处理样本 1: images/1-vehicle-odometer-reading.jpg

成功处理样本 2: images/2-vehicle-odometer-reading.jpg

成功加载 2 个训练样本

```
[{'messages': [{'role': 'user',
  'content': [{'type': 'text',
    'text': '你是一名汽车保险承保专家。这里有一张车辆里程表的图片。请从中提取关键信息。'}],
  {'type': 'image',
    'image': <PIL.Image.Image image mode=RGB size=426x320>}]},
 {'role': 'assistant',
  'content': [{'type': 'text',
    'text': '这张车辆里程表图片显示了以下关键信息：\n\n1. **总里程**：528,915公里\n2. **当前速度**：0公里/小时\n3. **当前时间**：19:18\n4. **当前温度**：+4.5°C\n5. **当前挡位**：停车挡（P）'}]}],
 {'messages': [{'role': 'user',
  'content': [{'type': 'text',
    'text': '你是一名汽车保险承保专家。这里有一张车辆里程表的图片。请从中提取关键信息。'}],
  {'type': 'image',
    'image': <PIL.Image.Image image mode=RGB size=800x445>}]},
 {'role': 'assistant',
  'content': [{'type': 'text',
    'text': '从这张车辆里程表的图片中，可以提取以下关键信息：\n\n1. **当前速度**：56.6英里/小时。 \n2. **总里程**：37756英里。 \n3. **当前时间**：11:14。 \n4. **当前温度**：10.5°C。 \n5. **当前挡位**：P（停车挡）。'}]}]]
```

CASE：Qwen2.5-VL 3B 视觉模型微调

第一个训练样本结构：
用户消息文本：你是一名汽车保险承保专家。这里有一张车辆里程表的图片。请从中提取关键信息。
助手回复：这张车辆里程表图片显示了以下关键信息：

- 1. ****总里程****：528,915公里
- 2. ****当前速度****：0公里/小时
- 3. ****当前时间****：19:18
- 4. ****当前温度****：+4.5°C
- 5. ****当前挡位****：停车挡（P）

训练前模型推理测试...
训练前模型输出：
从提供的图片中提取的关键信息如下：

- 1. ****速度表****：
 - 当前速度显示为**20**公里/小时。
- 2. ****里程数****：
 - 现在车辆的总行驶里程是**529891.5**公里。
- 3. ****车况****：
 - 行驶距离：**2600000.7**公里。
 - 制动测试：**19,181.3**公里。
- 4. ****油耗****：
 - 高速段燃油消耗为**44**升（平均）。
 - 城市

开始训练模型...

Unsloth: Model does not have a default image size - using 512

GPU = NVIDIA GeForce RTX 4090. 最大显存 = 23.65 GB.

3.926 GB 显存已使用.

==((====))== Unsloth - 2x faster free finetuning | Num GPUs used = 1

\\ /| Num examples = 2 | Num Epochs = 30 | Total steps = 30

0^0/ _/ \ Batch size per device = 2 | Gradient accumulation steps = 4

\ / Data Parallel GPUs = 1 | Total batch size (2 x 4 x 1) = 8

"-__-" Trainable parameters = 41,084,928/3,000,000,000 (1.37% trained)

`use\_cache=True` is incompatible with gradient checkpointing. Setting `use\_cache=False`

[30/30 00:26, Epoch 30/30]

Step	Training Loss
1	2.389500
2	2.397800
3	2.388600
4	2.320300
5	2.073300
6	1.748000
7	1.443700
8	1.226700
9	1.065100
10	0.919100
11	0.784000

CASE: Qwen2.5-VL 3B 视觉模型微调

Unsloth: Will smartly offload gradients to save VRAM!

训练用时: 69.923 秒

训练用时: 1.17 分钟

峰值显存使用: 3.926 GB

LoRA训练显存使用: 0.0 GB

显存使用率: 16.6%

LoRA显存使用率: 0.0%

训练后模型推理测试...

训练后模型输出:

从这张车辆里程表的图片中, 可以提取以下关键信息:

1. ****总里程****: 528,915公里。
2. ****当前速度****: 0公里/小时。
3. ****当前时间****: 19:18 (晚)。
4. ****当前温度****: +4.6°C。
5. ****当前挡位****: 停车挡。 <|im_end|>

保存LoRA适配器...

训练完成! 模型已保存到 car_insurance_lora_model 目录

Summary

Thinking: Qwen-VL多模态微调与纯文本SFT的差异?

1. 使用FastVisionModel而非FastLanguageModel

```
model, tokenizer = FastVisionModel.from_pretrained(  
    "Qwen2.5-VL-3B-Instruct",  
    use_gradient_checkpointing="unsloth"  
)
```

2. LoRA配置需同时微调视觉层和语言层

```
model = FastVisionModel.get_peft_model(  
    model,  
    finetune_vision_layers=True, # 微调图像编码器  
    finetune_language_layers=True, # 微调文本解码器  
    finetune_attention_modules=True,  
    finetune_mlp_modules=True,  
    r=16, lora_alpha=16,  
)
```

Summary

3. 数据格式: Excel(id, prompt, image, response) -> messages

```
conversation = {
```

```
    "messages": [
```

```
        {"role": "user", "content": [
```

```
            {"type": "text", "text": prompt},
```

```
            {"type": "image", "image": PIL.Image对象}
```

```
        ]},
```

```
        {"role": "assistant", "content": [{"type": "text", "text":
```

```
response}}]
```

```
}
```

4. 必须使用UnslothVisionDataCollator

```
trainer = SFTTrainer(
```

```
    ...,
```

```
    data_collator=UnslothVisionDataCollator(model, tokenizer),
```

```
    dataset_kwargs={"skip_prepare_dataset": True}, # 视觉模型特殊配置
```

```
)
```

5. 显存占用: 3B VL模型约4GB, 比7B纯文本还省 (因图像压缩后token较少)

何时用VL微调? => 当任务需要看图说话时, 如文档理解、图表分析、OCR增强。

蒸馏/微调效果评估

Thinking：训练完成后，如何系统性地评估不同方法的效果？

评估对比维度：

1. 微调前 vs 微调后定量对比

- 同一组测试问题，对比基座模型和微调模型的回答
- 量化指标：格式遵循、答案准确率、语言匹配
- 基座模型（微调前）通常用英文回答中文问题

2. 不同方法对比：纯SFT vs SFT+GRPO

- SFT模型：擅长格式遵循和知识回答，但推理能力有限
- GRPO模型：擅长推理和结构化输出，但可能牺牲流畅度
- 不同任务适合不同方法

3. 评估维度（4个核心指标）：

- a) 格式遵循能力：模型是否按要求的格式输出
 - 医疗场景：是否有结构化回答
 - 推理场景：是否包含<reasoning>和<answer>标签
- b) 答案准确率：基于关键词匹配的F1评分
- c) 推理链质量：推理过程的长度、分步性、结论性
- d) 语言匹配：回复语言是否与期望一致（中文/英文）

蒸馏/微调效果评估

评估指标类

class EvalMetrics:

1. 格式遵循能力评估

def format_compliance(response, expected_format):

if expected_format == "xml":

检查是否包含<reasoning>和<answer>标签

has_reasoning = re.search(r'<reasoning>.*</reasoning>', response)

has_answer = re.search(r'<answer>.*</answer>', response)

elif expected_format == "medical":

检查是否有结构化回答(长度>20或包含###)

2. 答案准确率评估 (简化版F1)

def answer_accuracy(response, reference):

字符级别的精确率和召回率

overlap = len(ref_chars & resp_chars)

f1 = 2 * precision * recall / (precision + recall)

model_comparison_eval.py

蒸馏/微调效果评估

```
# 3. 推理链质量评估
def reasoning_quality(response):
    # 检查推理长度(>50字)、分步指标、结论性指标
    step_indicators = ['首先', '然后', '因此', '1.', '2.']

# 4. 语言匹配评估
def language_match(response, expected="chinese"):
    # 统计中文字符占比

# 多模型对比评估
def compare_models(test_cases, model_results):
    # 对每个测试用例，分别评估各模型的回复
    # 输出各模型的综合评分对比
```

对比评估示例:

模型	格式遵循	答案准确	语言匹配	综合评分
基座模型 (微调前)	0.3	0.15	0.05	0.17
SFT微调模型	0.9	0.72	0.95	0.86
GRPO强化学习模型	0.95	0.68	0.88	0.84

打卡： Qwen-VL视觉模型微调

快来训练你的视觉模型，使用Qwen2.5-VL-3B 或其他尺寸的Qwen-VL模型

- 1) 准备微调数据集
- 2) 数据集格式转化
- 3) 配置unsloth，使用lora 对 Qwen2.5-VL进行模型微调
- 4) 使用微调后的模型进行测试



Thank You
Using data to solve problems